CS4433 – Moderated Learning Management System

Test Strategy

1. Introduction

1.1 Purpose

This document defines the testing strategy for the Moderated Learning Management System (LMS), a C++17 application developed to support academic administration, student enrolment, assignment submission, grading, moderation and result release.

The purpose of this strategy is to establish how the system will be verified and validated to ensure that it satisfies its functional and non-functional requirements.

Testing focuses on identifying defects, verifying individual components, checking interactions between system modules and evaluating whether the completed application meets the needs of its intended users.

1.2 Testing Objectives

The main objectives are to:

- **Verify that individual classes, functions and services operate correctly.**
- **Confirm that repositories communicate correctly with the SQLite database.**
- **Verify authentication and role-based access control.**
- **Test student enrolment and assignment submission.**
- **Verify the late submission penalty policy and individual extensions.**
- **Test the grading, moderation and result-release workflow.**
- **Verify that mark changes are recorded in the audit log.**
- **Evaluate input validation, error handling and database reliability.**
- **Measure automated test coverage against the project target.**

2. Testing Approach

The project follows a structured testing approach based on the V-model. Each development level is associated with a corresponding testing level.

Development activity	Corresponding testing activity
Requirements analysis	Acceptance testing
System design	System testing

Architectural design	**Integration testing**
Component design and implementation	**Unit testing**

Testing is performed at four main levels.

2.1 Unit Testing

Unit testing verifies individual components independently to ensure that their internal logic produces the expected results.

The following components are included:

- **AuthenticationService**
- **PasswordService**
- **CreateAccount**
- **ThreeDayGraduatedLatePenaltyPolicy**
- **LatePenaltyPolicy**
- **UserRepository**
- **CourseRepository**
- **DepartmentRepository**
- **EnrollmentRepository**
- **ExtensionRepository**

GoogleTest is used to implement and execute automated unit tests.

Examples include:

- **Verifying password hashing and password verification.**
- **Testing successful and unsuccessful authentication.**
- **Validating account creation.**
- **Testing late penalty calculations.**
- **Checking repository operations and invalid inputs.**

2.2 Integration Testing

Integration testing verifies that related components work correctly when connected.

The main integration areas include:

- **Authentication services and user repositories.**
- **Account creation services and database operations.**
- **Student enrolment and course repositories.**
- **Assignment and submission repositories.**
- **Extension management and submission grading.**
- **Submission grading, moderation and result release.**

SQLite is used to verify database interactions. Integration tests check that data is inserted, retrieved and updated correctly and that related operations maintain the expected workflow.

2.3 System Testing

System testing evaluates the complete LMS application as an integrated system.

The application is executed through its console interface using the different user roles:

- **Student**
- **Lecturer**
- **Head of Department**
- **Administrator**

System testing covers authentication, role-specific menus, course management, enrolment, assignment creation, submission, grading, moderation, result release and error handling.

Both normal and exceptional scenarios are considered.

2.4 Acceptance Testing

Acceptance testing evaluates whether the system provides the expected functionality from the perspective of its intended users.

The acceptance scenarios are based on the main system use cases.

Examples include:

- **A student logging in and enrolling in a course.**
- **A student submitting an assignment.**
- **A lecturer creating an assignment and grading submissions.**
- **A lecturer granting an individual extension.**

- **A Head of Department moderating and releasing results.**
- **An administrator managing users, departments and courses.**

Acceptance testing focuses on whether these activities can be completed correctly through the application.

3. Testing Tools and Environment

The following tools and technologies are used for testing:

Tool / Technology	Purpose
C++17	Main implementation language
GoogleTest	Automated unit and integration testing
CTest	Test execution and reporting
SQLite	Database testing and persistence verification
CMake	Project configuration and build management
GCC / MinGW	Compilation and linking
gcov	Collection of code coverage information
gcovr	Generation of code coverage summaries and HTML reports
VS Code	Development and debugging environment
Windows 11	Primary testing environment

Automated tests are built and executed using CMake and CTest.

The following commands are used:

cmake -S . -B build -G "MinGW Makefiles"

cmake --build build

ctest --test-dir build --output-on-failure

Code coverage is collected using a separate coverage build:

cmake -S . -B build-coverage -G "MinGW Makefiles" -DENABLE_COVERAGE=ON

cmake --build build-coverage

ctest --test-dir build-coverage --output-on-failure

gcovr --root . --filter 'src/(services|repositories)/' --print-summary --html-details coverage/index.html build-coverage

4. Test Design Techniques

The following test design techniques are used to select meaningful test cases.

4.1 Equivalence Partitioning

Input values are divided into groups expected to produce similar behaviour.

For example, login inputs can be divided into:

- **Valid username and password.**
- **Valid username with an incorrect password.**
- **Unknown username.**
- **Empty username.**
- **Empty password.**

Assignment submission inputs can be divided into valid assignment identifiers, invalid identifiers and assignments unavailable to the student.

4.2 Boundary Value Analysis

Boundary value analysis tests values at and around important limits.

It is particularly important for the late submission penalty policy.

The policy applies the following rules:

Submission time	Expected penalty
On or before the deadline	0%
1 day late	10% of maximum mark
2 days late	20% of maximum mark
3 days late	30% of maximum mark
More than 3 days late	Final mark is 0

The tests should include the deadline itself, the first day after the deadline, the third day after the deadline and the first day beyond the permitted three-day period.

Additional boundary tests include:

- **An awarded mark of zero.**
- **An awarded mark equal to the maximum mark.**
- **A negative mark.**
- **A maximum mark of zero or less.**
- **Invalid assignment and user identifiers.**

4.3 Negative Testing

Negative testing verifies that the system handles invalid operations safely.

Examples include:

- **Logging in with incorrect credentials.**
- **Entering invalid menu options.**
- **Entering alphabetic input where a number is required.**
- **Submitting an assignment that is not available to the student.**
- **Attempting to create an account with an invalid role.**
- **Attempting database operations with invalid identifiers.**

The expected behaviour is that the application rejects invalid operations and provides an appropriate message without terminating unexpectedly.

5. Entry and Exit Criteria

5.1 Entry Criteria

Testing may begin when:

- **The relevant source code has been implemented.**
- **Required dependencies and libraries are installed.**
- **The project can be configured and compiled.**
- **The SQLite database schema is available.**
- **Test cases and expected results have been defined.**
- **The required test accounts and data are available.**

5.2 Exit Criteria

A testing stage may be considered complete when:

- **The planned tests for that stage have been executed.**
- **Actual results have been compared with expected results.**
- **Failed tests have been investigated and recorded.**
- **Critical defects have been resolved or formally documented.**
- **Automated tests execute successfully.**
- **The automated line coverage target of at least 70% has been achieved.**

- **Test execution evidence has been recorded.**

A successful automated test run does not, by itself, demonstrate that every system or acceptance test has passed.

6. Test Coverage Requirements

The project establishes a minimum automated line coverage target of 70% for the selected business logic.

The following coverage areas are prioritised:

Component	Testing priority
Authentication and password services	High
Late penalty policy	High
Account creation	High
Submission workflow	High
Extension management	High
Database repositories	High
Input validation	High
Console menu presentation	Medium

The late penalty policy and authentication services receive particular attention because errors in these areas could affect account security and academic results.

7. Test Execution and Results

7.1 Automated Test Results

The automated test suite was executed using CTest.

The recorded results were:

Measurement	Result
Tests passed	156
Tests failed	0
Total execution time	5.62 seconds
Line coverage	72.8%
Function coverage	91.1%
Branch coverage	40.7%

The measured line coverage was 982 out of 1,349 lines.

The project therefore achieved its minimum automated line coverage target of 70%.

7.2 Authentication Test Results

The authentication test suite reported six successful tests:

1. **Login with correct credentials returns a user.**
2. **Login with an incorrect password returns null.**
3. **Login with an unknown username returns null.**
4. **Login with an empty username returns null.**
5. **Login with an empty password returns null.**
6. **Login with different username letter casing succeeds.**

These tests verify important authentication scenarios, including invalid credentials and empty input.

7.3 Manual System Test Results

Manual system testing was performed using the compiled LMS executable.

The following activities were demonstrated successfully:

- **Student, Lecturer and Administrator login.**
- **Display of role-specific menus.**
- **Logout and return to the main menu.**
- **Student viewing available courses.**
- **Student enrolment in courses.**
- **Administrator creation of a Student account.**
- **Login using the newly created Student account.**
- **Student viewing assignments for enrolled courses.**
- **Student submitting an assignment.**
- **Display of the individual extension deadline for a student with an approved extension.**
- **Display of the original deadline for a student without an extension.**

The extension test confirmed that Assignment 4 displayed the revised deadline of 15 November 2026 for the student with an approved extension, while another student without an extension saw the original deadline of 10 November 2026.

The remaining planned manual system and acceptance tests have not been recorded as completed in this strategy.

8. Defect Management

Defects identified during testing are recorded and prioritised according to their impact.

The following severity levels are used:

Severity	Description
Critical	Prevents the system from operating or compromises essential system security or data integrity.
High	Prevents an important feature from functioning correctly.
Medium	Causes incorrect behaviour in a feature but does not prevent the entire system from operating.
Low	Minor issue affecting presentation or usability.

Each defect record should contain:

- **Defect identifier.**
- **Description of the problem.**
- **Steps required to reproduce it.**
- **Expected result.**
- **Actual result.**
- **Severity.**
- **Root cause, where identified.**
- **Corrective action.**
- **Retest result.**

After a defect has been corrected, the relevant test case must be executed again to verify the correction.

9. Known Issues and Limitations

The following issues were identified or remain areas for further verification:

Issue	Description	Status
Invalid assignment date	Assignment 2 displayed an invalid due date value of y.	Requires investigation
Resubmission behaviour	Multiple submissions for the same student and assignment require clarification against the intended resubmission requirements.	Requires verification
Test coverage	Branch coverage remains at 40.7%, indicating that additional branch testing may be beneficial.	Improvement required

Remaining manual tests	**Not all planned system and acceptance scenarios have been executed.**	**Outstanding**

These issues should be considered when evaluating the final system.

10. Continuous Integration

Automated testing is intended to be integrated into the project's development workflow using GitHub Actions.

The planned pipeline will:

1. **Check out the project repository.**
2. **Configure the CMake build.**
3. **Compile the application and test targets.**
4. **Execute automated tests using CTest.**
5. **Report build or test failures.**

This will help identify defects whenever changes are introduced into the project.

11. Conclusion

This test strategy establishes a structured approach for verifying and validating the Moderated Learning Management System.

The strategy combines unit, integration, system and acceptance testing, supported by automated testing tools and manual execution scenarios.

The recorded automated results demonstrate that 156 tests passed and that the project achieved 72.8% line coverage, exceeding the minimum target of 70%.

Manual testing has also demonstrated important application functionality, including authentication, role-specific access, course enrolment, account creation, assignment submission and individual extension deadline handling.

Although the completed tests provide evidence of the system's functionality, the outstanding issues and unexecuted scenarios must remain documented. Further testing and defect resolution would improve confidence in the final product.